Chapitre 4 — REST et SOAP

Notes essentielles — Master Ingénierie Informatique

Objectifs de la séance

- Définir REST comme style architectural
- Concevoir des URLs RESTful cohérentes
- Décomposer des requêtes/réponses REST réelles
- Décrire la structure d'un message SOAP
- Comparer REST et SOAP et argumenter un choix

Plan de la séance (4h)

- Qu'est-ce que REST ? (30 min)
- Structure des URLs REST (30 min)
- Exemples REST complets (30 min)
- Pause (15 min)
- Codes de réponse en contexte REST (20 min)
- Introduction à SOAP (25 min)
- Caractéristiques clés de SOAP (20 min)
- REST vs SOAP : comparaison (35 min)
- Quand choisir REST ou SOAP + exercices (15 min)

Qu'est-ce que REST ?

- Representational State Transfer
- Un style architectural, pas un protocole
- Formulé par Roy Fielding en 2000
- S'appuie sur des standards existants (HTTP)
- Deux API REST peuvent être totalement différentes techniquement

Les trois principes de REST

- Absence d'état (statelessness) — hérité directement de HTTP
- Orientation ressource — tout est modélisé comme une ressource
- Interface uniforme — mêmes méthodes HTTP pour toutes les ressources

Interface uniforme — exemple

- Approche non-REST : POST /annulerCommande?id=57
- Approche REST : DELETE /commandes/57
- REST réutilise systématiquement le même vocabulaire
- Prévisible, sans mémoriser des noms de fonctions arbitraires

Anatomie d'une URL REST

- URL de base : https://api.example.com
- Ressource (collection) : /users, au pluriel
- Identifiant : /123, désigne une instance précise
- Convention : minuscules, pluriel, pas de verbe dans l'URL

Explorer les variations d'URL

- GET /users : la collection complète
- GET /users/2 : un utilisateur précis
- GET /users/2/pets : une sous-ressource (relation)
- GET /users?gender=female : filtrage via la chaîne de requête

Le piège de conception à éviter

- /users/pages/1 mélange ressource et pagination
- Préférer la chaîne de requête pour filtrer/trier/paginer
- Réserver le chemin à la hiérarchie réelle des ressources
- La flexibilité de REST est aussi une responsabilité de conception

Exemple REST — GET

- GET /users renvoie un tableau JSON d'utilisateurs
- Accept: application/json demande le format
- Content-Type: application/json le confirme en retour
- Code de statut : 200 OK

Exemple REST — POST

- POST /users crée une nouvelle ressource
- Le client peut envoyer en XML et demander du JSON en retour
- Content-Type et Accept sont indépendants
- Code de statut : 201 Created, avec en-tête Location

Exemple REST — PUT et DELETE

- PUT /users/3 remplace intégralement la ressource — 204 No Content
- DELETE /users/2 supprime la ressource — 204 No Content
- PUT peut aussi créer si le client connaît déjà l'URI
- PUT et DELETE sont tous deux idempotents

Exemple REST — PATCH

- PATCH /users/1 modifie partiellement une ressource
- RFC 5789, standardisée bien après les autres méthodes (2010)
- Non idempotente en général

Codes de statut en contexte REST

- 200 OK — GET réussi
- 201 Created — POST réussi
- 404 Not Found — ressource inexistante
- 400 Bad Request — corps JSON malformé
- 500 Internal Server Error — bug côté serveur

Introduction à SOAP

- Simple Object Access Protocol
- Un protocole, pas seulement un style
- Exclusivement au format XML
- Peut s'appuyer sur HTTP, mais aussi SMTP
- Antérieur à REST (1998-2000 vs 2000)

Structure d'un message SOAP

- Envelope : encapsule tout le message (racine XML)
- Header (optionnel) : métadonnées, ex. authentification
- Body (obligatoire) : la donnée réelle de la requête/réponse

Exemple de message SOAP

- Envelope contient un Header avec un jeton d'authentification
- Le Body contient l'opération demandée (ex. GetUserDetails)
- Requête et réponse ont la même structure générale
- L'opération est un élément XML nommé arbitrairement

Caractéristiques clés de SOAP

- Basé sur un protocole strict — décrit par un WSDL
- Basé sur XML — indépendant de la plateforme
- WS-Security — sécurité intégrée au message lui-même
- Peut être stateful — contrairement à REST, toujours stateless

REST vs SOAP — nature

- Nature : style architectural (REST) vs protocole strict (SOAP)
- Format : multiple JSON/XML (REST) vs exclusivement XML (SOAP)
- Complexité : plus simple (REST) vs plus complexe (SOAP)

REST vs SOAP — communication

- État : toujours stateless (REST) vs stateless ou stateful (SOAP)
- Opérations : méthodes HTTP standards vs opérations arbitraires (WSDL)
- Description : documentation informelle/OpenAPI vs WSDL formel

REST vs SOAP — performance

- REST généralement plus rapide (cache HTTP facilité)
- SOAP plus lent (traitement XML, surcoût WS-Security)
- REST : GET + URL unique par ressource = cache possible
- SOAP : tout transite souvent en POST vers une seule URL

REST vs SOAP — cas d'usage

- REST : applications web/mobiles, réseaux sociaux, e-commerce
- SOAP : secteur bancaire, passerelles de paiement, télécoms
- SOAP : conformité ACID pour les transactions inter-systèmes

Quand choisir REST ?

- Service simple, scalable et flexible
- Plusieurs formats de données à supporter
- Vitesse de développement et performance prioritaires

Quand choisir SOAP ?

- Sécurité avancée intégrée au message (WS-Security)
- État à maintenir à travers plusieurs requêtes
- Standards stricts de transaction requis

Exercices de la séance

- Classer des scénarios : REST ou SOAP ?
- Corriger une URL REST mal conçue
- Identifier les composants d'un message SOAP

Synthèse

- REST : style architectural — stateless, orienté ressource, interface uniforme
- La conception d'URLs REST doit rester cohérente et non ambiguë
- SOAP : protocole strict, XML, sécurité intégrée, peut être stateful
- Le choix est un arbitrage simplicité/performance vs rigueur/conformité

Prochain chapitre

- Chapitre 5 : GraphQL et gRPC
- Deux alternatives modernes nées des limites de REST
- GraphQL : le client compose sa requête
- gRPC : performance et format binaire